```
    BaseType& expected,BaseType new_value,
    memory_order order = std::memory_order_seq_cst) noexcept;
bool compare_exchange_strong(
    BaseType& expected,BaseType new_value,
    memory_order success_order,memory_order failure_order)
    volatile noexcept;
bool compare_exchange_strong(
    BaseType& expected,BaseType new_value,
    memory_order success_order,memory_order failure_order)
noexcept;
```

Preconditions

`failure_order` shall not be `std::memory_order_release` or `std::memory_order_acq_rel`.

Effects

Atomically compares `expected` to the value stored in `*this` using bitwise comparison and stores `new_value` in `*this` if equal; otherwise updates `expected` to the value read.

Returns

`true` if the existing value of `*this` was equal to `expected, false` otherwise.

Throws

Nothing.

Note

The three-parameter overload is equivalent to the four-parameter overload with `success_order==order` and `failure_order==order`, except that if `order` is `std::memory_order_acq_rel`, then `failure_order` is `std::memory_order_acquire`, and if `order`